EE533 LAB7

Group 17
Pavneet Singh Banga, Patrick Sheng, Shishir Suresh Kumar

Github Link: [Patypal10/EE533_lab8: EE533 lab8 SP26]

HOST ARM CPU CODE

C CODE:

Handshaking registers: cpu_ready , packet_ready
We are polling packer_ready until it is 1 to begin data manipulation

```
int main() {
    register int cpu_ready asm("r1");
    register int packet_ready asm("r2");
    int data;

    while (1) {

        // Step 1: Assert CPU_READY signal
        cpu_ready = 1;

        // Step 2: Poll until FIFO asserts PACKET_READY
        while (packet_ready == 0) {
            // busy-wait
        }

        // Step 3: Deassert CPU_READY (now processing)
        cpu_ready = 0;

        // Step 4: Read 64-bit data from FIFO
        data = packet_ready;

        // Step 5: Increment and write back
        data = data + 1;
    }
return data;
}
```

Assembly Code:

```
        .cpu arm7tdmi
        .arch armv4t
        .fpu softvfp
        .eabi_attribute 20, 1
        .eabi_attribute 21, 1
        .eabi_attribute 23, 3
        .eabi_attribute 24, 1
        .eabi_attribute 25, 1
        .eabi_attribute 26, 1
        .eabi_attribute 30, 6
        .eabi_attribute 34, 0
        .eabi_attribute 18, 4
        .file   "CPU_code2.c"
        .text
        .align  2
        .global main
        .syntax unified
        .arm
        .type   main, %function
main:
        @ Function supports interworking.
        @ args = 0, pretend = 0, frame = 8
        @ frame_needed = 1, uses_anonymous_args = 0
        @ link register save eliminated.
        str     fp, [sp, #-4]!
        add     fp, sp, #0
        sub     sp, sp, #12
.L3:
        mov     r1, #1
        nop
.L2:
        mov     r3, r2
        cmp     r3, #0
        beq     .L2
        mov     r1, #0
        str     r2, [fp, #-8]
        ldr     r3, [fp, #-8]
        add     r3, r3, #1
        str     r3, [fp, #-8]
        b       .L3
        .size   main, .-main
        .ident  "GCC: (Arm GNU Toolchain 15.2.Rel1 (Build arm-15.86)) 15.2.1 20251203"
```

The part that is important for our assembler is put in bold.
The assembler from Lab6 converts this into binaries for the CPU.

Later in GPU integration where we our doing data + 1 in the C code, we just put a call to the GPU

Network Memory Verilog Code:

network_mem.v M

C: > Users > da_sambanda > Desktop > SSK > EE533labs > lab8 > git_lab8 > EE533_lab8_v2 > network_mem.v

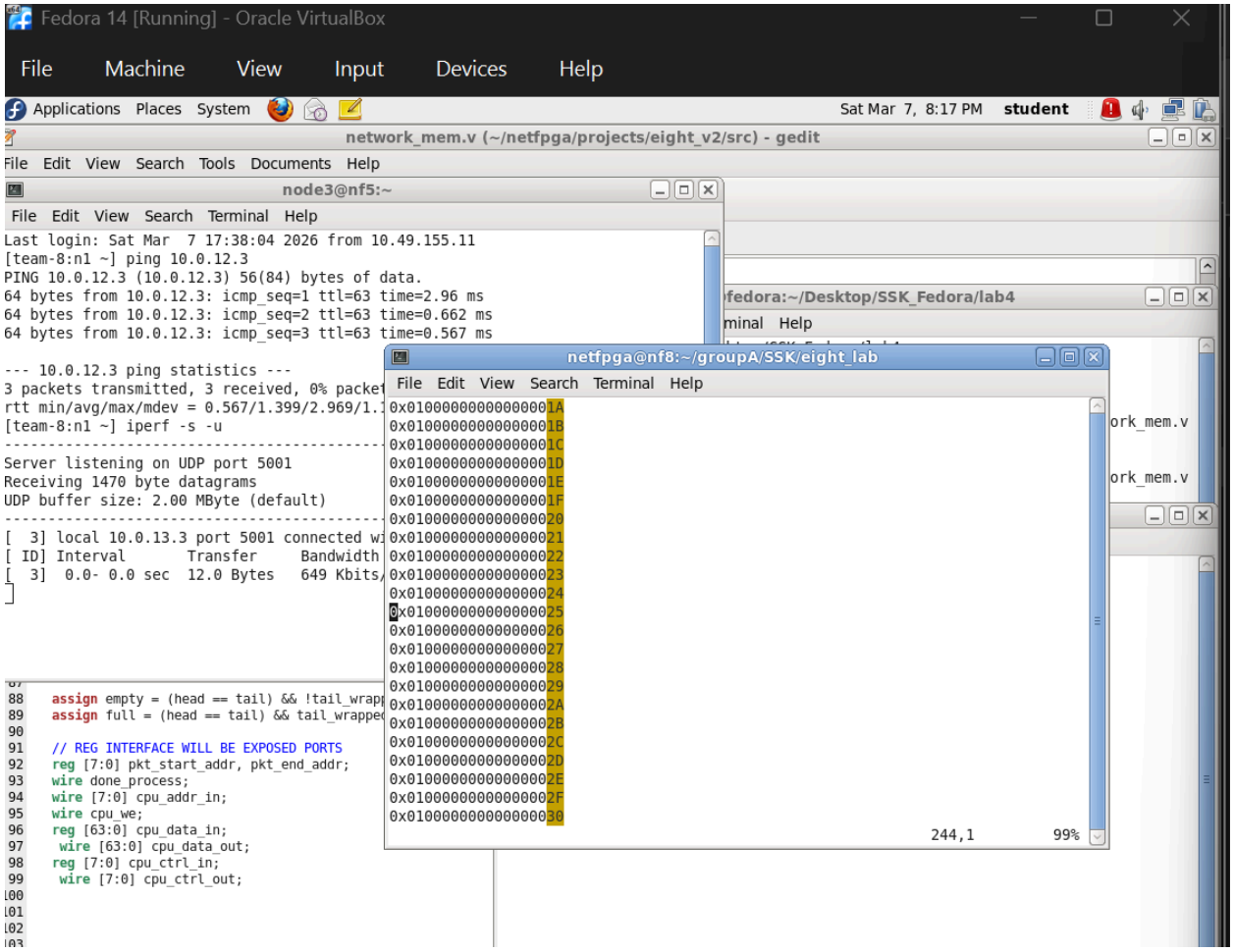
```
3   module network_mem

100
101     assign in_rdy = (state == START) || (((state == CAPTURE_HEADER) || (state == CAPTURE_PAYLOAD)) && !set_end_addr);
102     assign out_wr = out_rdy && read_req;
103
104     // FOR TESTING
105     wire [31:0] mem_addr, command_reg;
106     wire [31:0] mem_data_lsb, mem_data_msb, mem_ctrl;
107     wire [31:0] pkt_start_debug, pkt_end_debug;
108     wire [31:0] flag;
109     wire [31:0] first_data_debug;
110     wire [31:0] head_ptr_debug, tail_ptr_debug;
111     wire [31:0] payload_cycles_debug;
112     wire [31:0] curr_state_debug;
113     wire [31:0] out_wr_ct_debug;
114
115     assign mem_data_lsb = cpu_data_out[31:0];
116     assign mem_data_msb = cpu_data_out[63:32];
117     assign mem_ctrl = {{28'd0}, cpu_ctrl_out};
118     assign pkt_start_debug = pkt_start_addr;
119     assign pkt_end_debug = pkt_end_addr;
120     reg [31:0] pkts_ct, pkts_ct_next;
121     assign flag = pkts_ct;
122     reg [31:0] first_data_reg;
123     assign first_data_debug = first_data_reg;
124     assign head_ptr_debug = {{24'd0}, head};
125     assign tail_ptr_debug = {{24'd0}, tail};
126     reg [31:0] payload_cycles_reg;
127     assign payload_cycles_debug = payload_cycles_reg;
128     assign curr_state_debug = {{29'd0}, state};
129     reg [31:0] out_wr_ct_reg;
130     assign out_wr_ct_debug = out_wr_ct_reg;
131
132     // assign state_out = state;
133     // assign start_addr_out = pkt_start_addr;
134     // assign end_addr_out = pkt_end_addr;
135     assign done_process = 1;
136     assign cpu_addr_in = (command_reg[2:0] == 3'b001) ? mem_addr : 8'hff;
137     assign cpu_we = 0;
```

Working of Network Memory Block:

- The network_mem module is a packet capture and forwarding buffer inserted into the NetFPGA pipeline. It captures incoming packets into a BRAM-based circular FIFO, and forwards them downstream after processing.
- The module sits between the Router and Output Queue so I can receive and process all the data sent in the pipeline.
- The BRAM is dual ported. One of the ports is permanently accessed by the CPU. While the other port is multiplexed between the input FIFO, GPU and output port.
- The FIFO uses a BRAM memory in a circular fashion with head and tail pointers.
- The Network Memory currently implemented uses a FSM Architecture:
- The FSM has five states:
- **START** — Idle, waiting for a new packet. Remains here until the beginning of a NetFPGA module header is detected. On detection, transitions to CAPTURE_HEADER and records the packet start address in BRAM.
- **CAPTURE_HEADER** — Captures all NetFPGA module header words followed by the Ethernet/IP/TCP header words into BRAM. Transitions to CAPTURE_PAYLOAD when real packet data begins and resets the word counter.
- **CAPTURE_PAYLOAD** — Captures all payload words into BRAM, incrementing the word counter each cycle. Continues until the end-of-packet marker arrives, at which point the end address is recorded and the FSM transitions to PROCESS.
- **PROCESS** — This informs the CPU via a memory mapped method. The CPU processes the data and sends a process_done signal to move to next state.
- **FLUSH** — Reads the captured packet back out word by word, forwarding it downstream. Continues until the entire packet has been sent, then returns to START for the next packet.

Output Results:

Writing a counter output in the TCP packet:



Here we are performing a simple data manipulation where the payload data is substituted with a counter information.

Script for Sending TCP IP Data Over Node 0:

network_mem.v M sender_perl.pl

C: > Users > da_sambanda > Desktop > SSK > Shared_folder > lab8 > sender_perl.pl

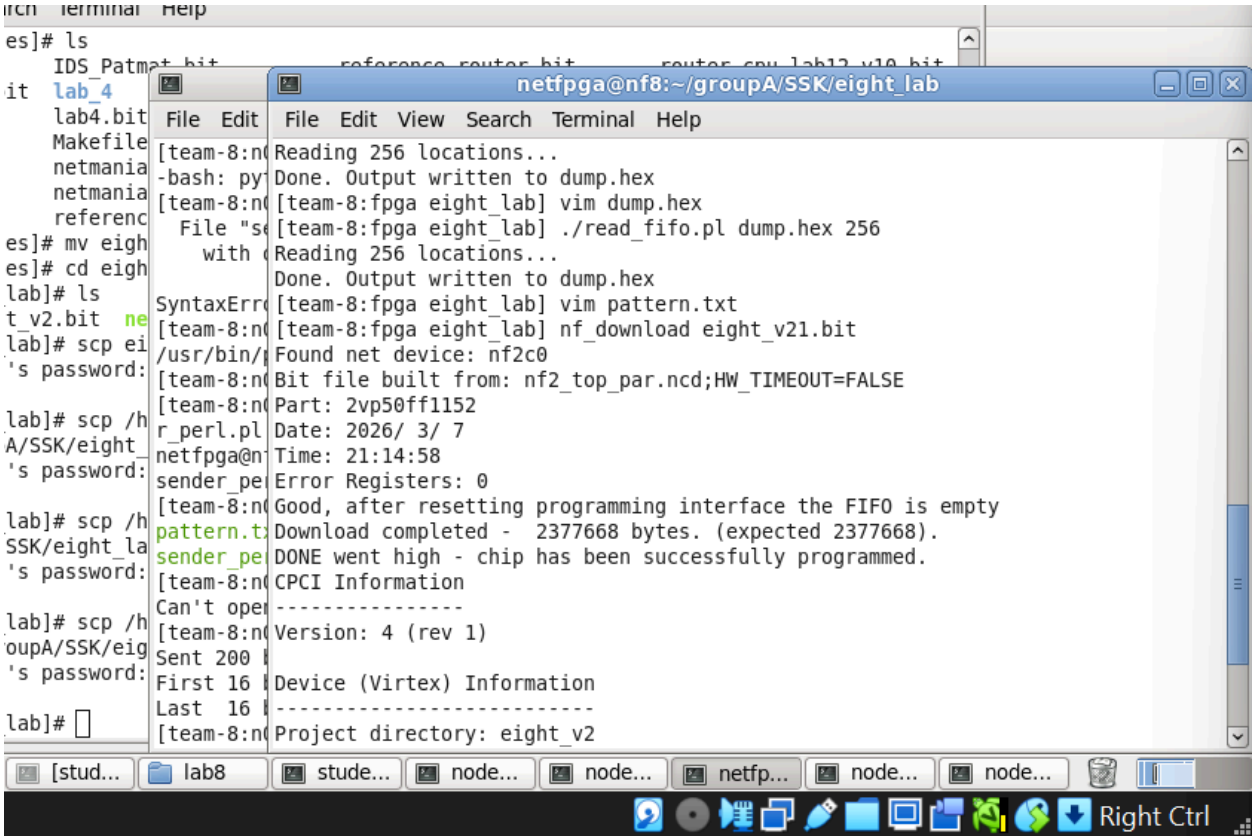
```perl
#!/usr/bin/perl
use strict;
use warnings;
use IO::Socket::INET;

# Check arguments
if (@ARGV != 2) {
    print "Usage: perl sender.pl <input_file> <dest_ip>\n";
    exit 1;
}

my ($filename, $dest_ip) = @ARGV;
my $dest_port = 5005;

# Read 200 lines from file
open(my $fh, '<', $filename) or die "Cannot open file: $!";
my @lines;
while (my $line = <$fh>) {
    chomp $line;
    push @lines, $line;
    last if @lines == 200;
}
close($fh);

if (@lines < 200) {
    print "Error: file has only " . scalar(@lines) . " lines, need 200\n";
    exit 1;
}

# Convert hex values to bytes
my $data = '';
foreach my $line (@lines) {
    $data .= chr(hex($line));
}

# Send as single UDP packet
my $socket = IO::Socket::INET->new(
    PeerAddr => $dest_ip,
    PeerPort => $dest_port,
    Proto    => 'udp'
) or die "Cannot create socket: $!";
```

Data Sent Over the Network:

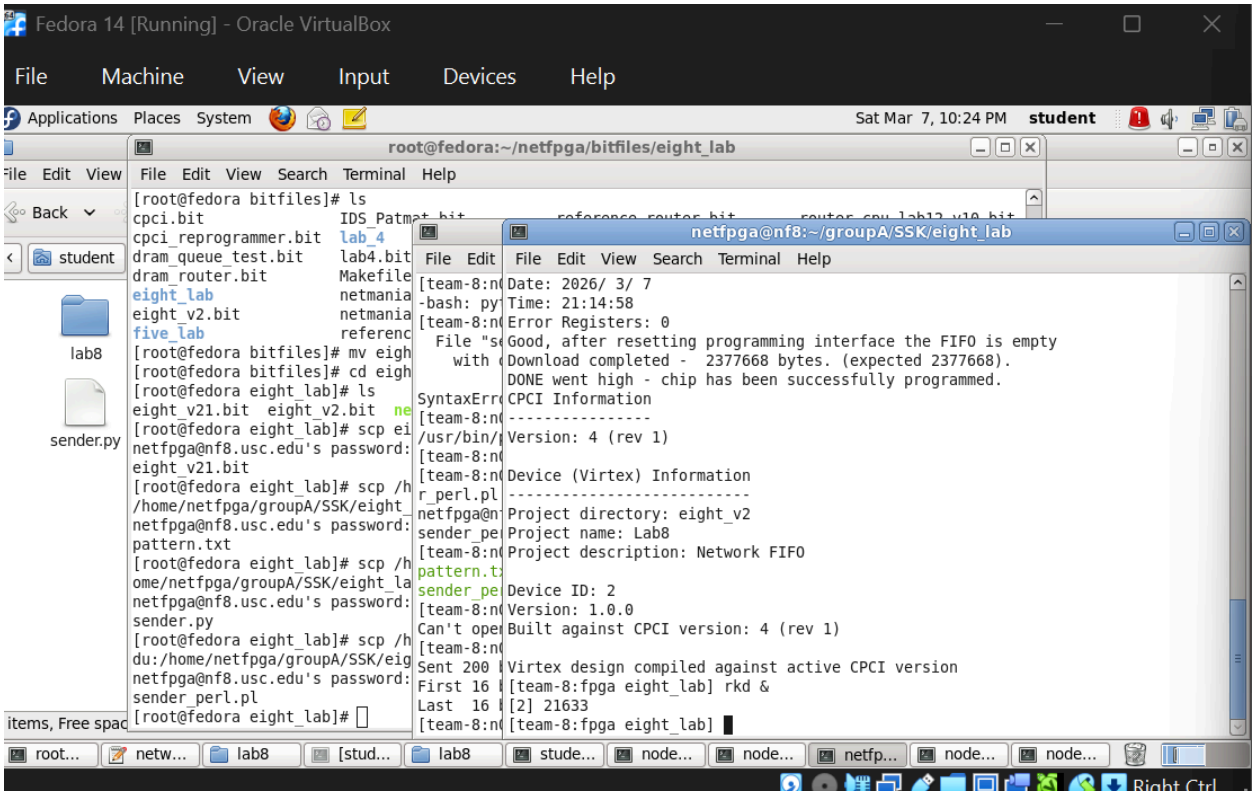
```
netfpga@nf8:~/groupA/SSK/eight_lab
File  Edit  View  Search  Terminal  Help
0x00
0x00
0x00
0x00
0x11
0x11
0x11
0x11
0x11
0x11
0x11
0x11
0x22
0x22
0x22
0x22
0x22
0x22
0x22
0x22
0x33
0x33
0x33
                                                    27,1          2%
```

Procedure For Output:

1. Burn the bitfile onto the netfpga.



2. Run the rkd &


Fedora 14 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Applications Places System
Sat Mar 7, 10:24 PM student
root@fedora:~/netfpga/bitfiles/eight_lab
File Edit View Search Terminal Help
[root@fedora bitfiles]# ls
cpci.bit IDS_Patmat.bit
cpci_reprogrammer.bit lab_4
dram_queue_test.bit lab4.bit
dram_router.bit Makefile
eight_lab netmania
eight_v2.bit netmania
five_lab referenc
[root@fedora bitfiles]# mv eigh
[root@fedora bitfiles]# cd eigh
[root@fedora eight_lab]# ls
eight_v21.bit eight_v2.bit ne
[root@fedora eight_lab]# scp ei
netfpga@nf8.usc.edu's password:
eight_v21.bit
[root@fedora eight_lab]# scp /h
/home/netfpga/groupA/SSK/eight_
netfpga@nf8.usc.edu's password:
pattern.txt
[root@fedora eight_lab]# scp /h
ome/netfpga/groupA/SSK/eight_la
netfpga@nf8.usc.edu's password:
sender.py
[root@fedora eight_lab]# scp /h
du:/home/netfpga/groupA/SSK/eig
netfpga@nf8.usc.edu's password:
sender_perl.pl
[root@fedora eight_lab]#
netfpga@nf8:~/groupA/SSK/eight_lab
File Edit View Search Terminal Help
Date: 2026/ 3/ 7
Time: 21:14:58
Error Registers: 0
Good, after resetting programming interface the FIFO is empty
Download completed - 2377668 bytes. (expected 2377668).
DONE went high - chip has been successfully programmed.
CPCI Information

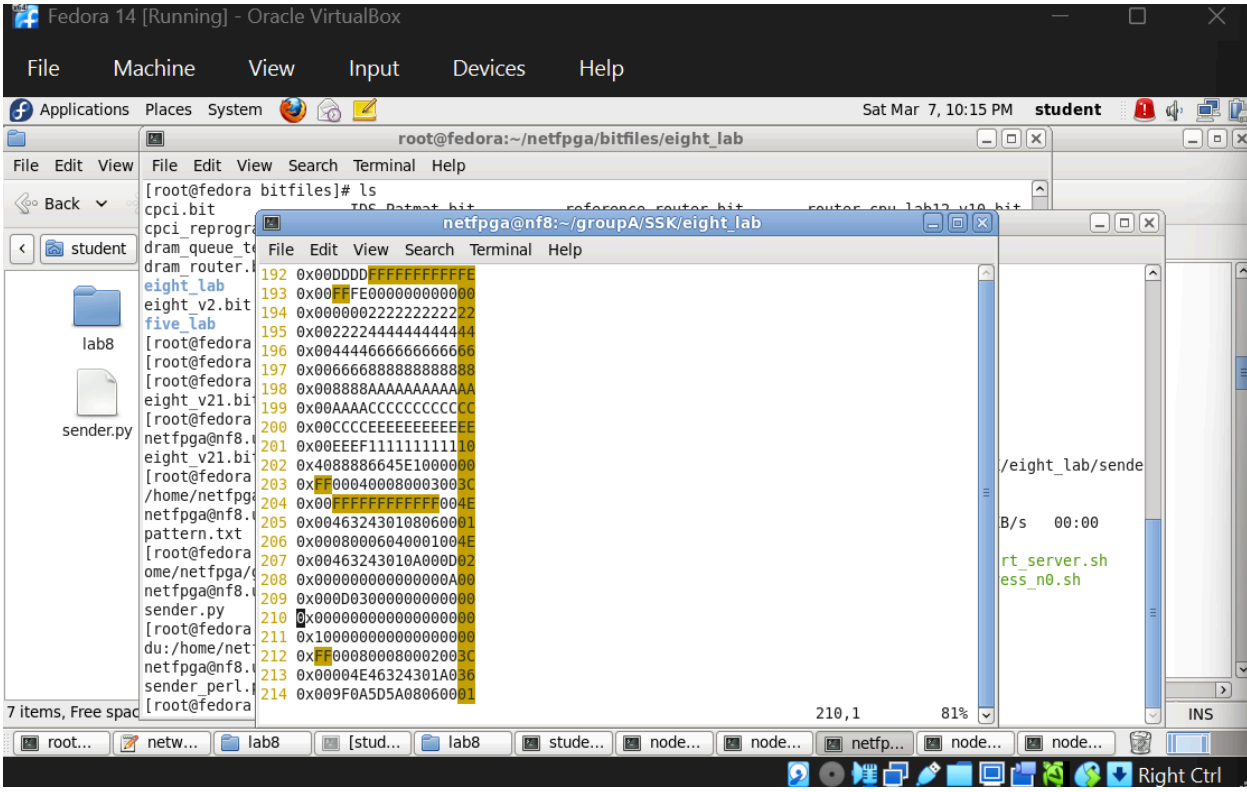
Version: 4 (rev 1)
Device (Virtex) Information

Project directory: eight_v2
Project name: Lab8
Project description: Network FIFO
Device ID: 2
Version: 1.0.0
Built against CPCI version: 4 (rev 1)
Virtex design compiled against active CPCI version
[team-8:fpga eight_lab] rkd &
[2] 21633
[team-8:fpga eight_lab]
lab8
sender.py

3. Use the Perl scirpt to send the data over the TCP network

```
node3@nf9:~/ssk
File  Edit  View  Search  Terminal  Help
[team-8:n0 ssk] python3 sender.py pattern.txt 10.0.13.3
-bash: python3: command not found
[team-8:n0 ssk] python sender.py pattern.txt 10.0.13.3
  File "sender.py", line 6
    with open(filename, 'r') as f:
            ^
SyntaxError: invalid syntax
[team-8:n0 ssk] which python
/usr/bin/python
[team-8:n0 ssk] vim sender.py
[team-8:n0 ssk] scp netfpga@nf8.usc.edu:/home/netfpga/groupA/SSK/eight_lab/sende
r_perl.pl .
netfpga@nf8.usc.edu's password:
sender_perl.pl                                100% 1193     1.2KB/s   00:00
[team-8:n0 ssk] ls
pattern.txt     sender.py            start_server_nic.sh     start_server.sh
sender_perl.pl  start_server_ids.sh  start_server_router.sh  stress_n0.sh
[team-8:n0 ssk] perl sender.pl pattern.txt 10.0.13.3
Can't open perl script "sender.pl": No such file or directory
[team-8:n0 ssk] perl sender_perl.pl pattern.txt 10.0.13.3
Sent 200 bytes to 10.0.13.3:5005
First 16 bytes: 00 00 00 00 00 00 00 00 11 11 11 11 11 11 11 11
Last  16 bytes: 77 77 77 77 77 77 77 77 88 88 88 88 88 88 88 88
[team-8:n0 ssk] perl sender_perl.pl pattern.txt 10.0.13.3
```

4. Use the read script to read the data from the BRAM - TCP DUMP:



5. Original Sent data:

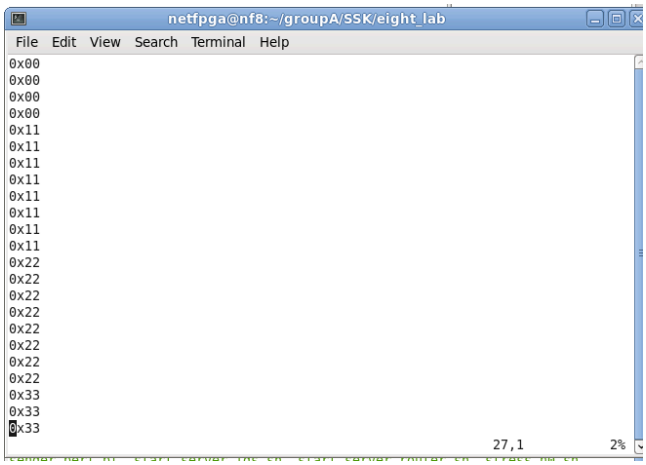


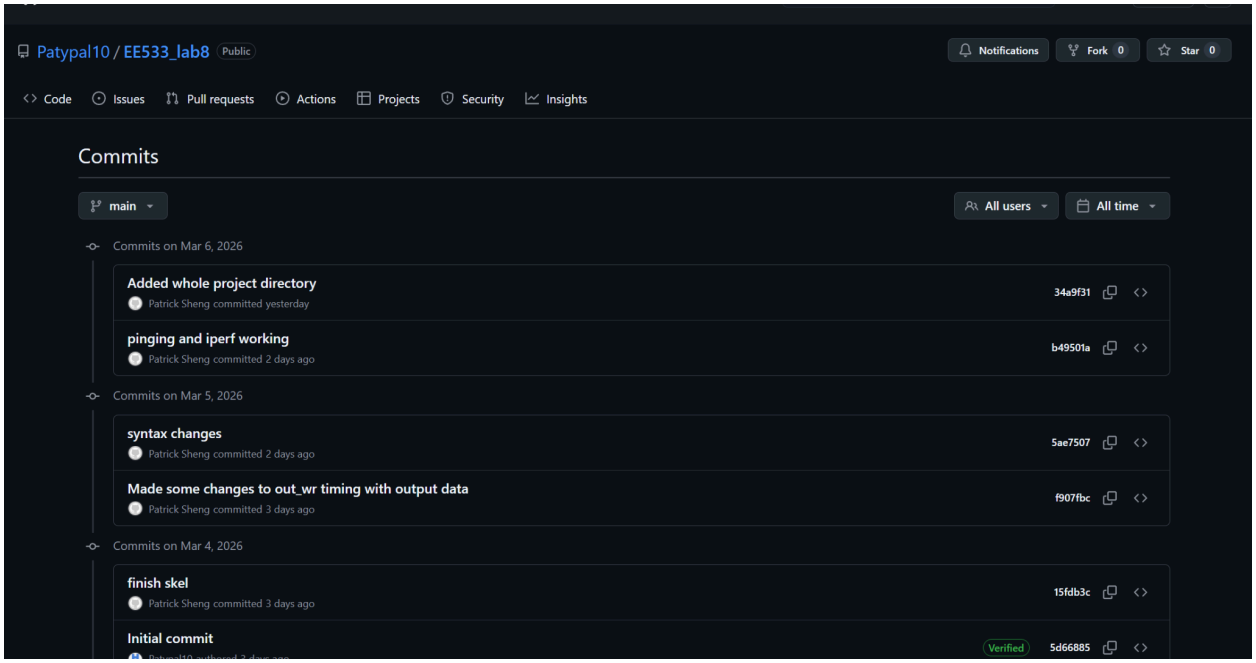
FIG: Comparing Sent and Received Data

Results:

Apart from the header data, we notice that the payload data is doubled consistently, proving that our data has been manipulated within our netfpga. Please find the input and output data to compare. The top is the TCP data dump extracted from the BRAM. The bottom is the original data sent on the network using the perl script.

We send a series of data bits increasing consecutively. We made a code to double the value in the netfpga. We notice that the output is consistently doubled to 00, 44, 88 etc.

Github Commit History:



Group Member Contributions:

1. Patrick Sheng worked on the Network FIFO which can communicate with the upstream and downstream to acquire and send out data.
2. Shishir Suresh Kumar integrating the DPU, GPU and memory mapped communication between the SOC components.
3. Pavneet implemented the GPU centric tensors core for parallel execution and parsed the hex code required for deploying in the DPU and GPU.

With incremental efforts from all team members, we were able to use the netfpga to successfully process the network data and output it back on the network.